

CAP, Eventual, and CRDT



CUHK

CAP theorem

- Fault model now considers **network partitioning**
 - A very specific fault that some are still connecting, but some are not
- Tradeoff on
 - Choosing C over A
 - Your request may timeout
 - Choosing A over C
 - Your request returns
 - But the results might not be the latest or consistent

SAN FRANCISCO, Oct 20 (Reuters) - Amazon.com cloud service returned to normal operations on Monday afternoon, the company said, after an internet outage that caused global turmoil among thousands of sites, including some of the web's most popular apps like Snapchat and Reddit.

Still, Amazon ([AMZN.O](#))  said some AWS services had a backlog of messages that would take a few hours to process.

Amazon says AWS cloud service back to normal after outage disrupts businesses worldwide

By Greg Bensinger, Shubham Kalra and Deborah Mary Sophia

October 21, 2025 10:48 PM GMT+8 · Updated October 21, 2025



Aa



Consistency Model revisited

- Because of CAP
 - Internet companies prefer to keep A, so sacrifice C
- From Strong to Weak
 - From no divergence across replicas to allowing divergence
 - Linearizable consistency (a.k.a. strong consistency)
 - Sequential consistency (=total ordering + replication)
 - Causal consistency (strongest level in the presence of P)
 - FIFO consistency
 - **Eventual consistency**

Eventual Consistency

- Guarantee:
 - Eventual no lost updates
 - Eventually, your updates will be propagated to all replicas
 - But allow clients to see inconsistent intermediate result
 - If no more updates
 - Eventually, the replicas are identical
 - But in any order (not even FIFO client order)

Eventual Consistency Application

- Amazon's shopping-cart implementation
- 2-phase set $S = A + R$
 - Add(X):
 - $A.insert(X)$
 - Del(X):
 - $R.insert(X)$
 - Merge(S1, S2):
 - $= S1.A \cup S2.A$
 - $= S1.R \cup S2.R$
 - View():
 - Return $(A - R)$
- Now, generalized as CRDT
 - Conflict-Free Replicated Data Type

R1	R2	R3
Add(x)		
Add(y)		
Merge()		
Add(z)		
	View-> (x,y)	
	Del(x); Add(q)	
View->(x,y,z)		
Merge()		
View->(y,z,q)		

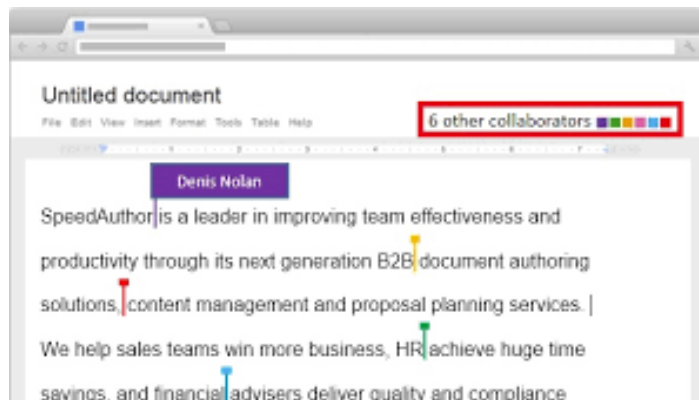
CRDT

- Associative *Late arrival*
Commutative vs *Reordered arrival*
Idempotence *Duplicated arrival*
- CmRDT (operation-based)
 - **C**ommutative replicated data types (CRDT ^_^')
 - Exchange the operations
- CvRDT (state-based)
 - **C**onvergent replicated data types (CRDT again ^_^")
 - Exchange the (delta) states

E.g., 5 (+1) (+2) (+3)
Alice $((5 + 1) + 2) + 3 = 11$
- user-viewable states: 6, 8, **11**
Bob $((5+3) + 2) + 1 = 11$
- user-viewable states: 8, 10, **11**

CRDT

- Is `s1.append(string s2)` ACI?
- If not, carry out operation transform to something like:
 - `s1.append(string s2) {`
 - Check their vector clock
 - Late one:
 - `insert_pos = earlier's one end pos + late's one length ...`
 - Some operations can carry out such “operation transform”; some don't



Properties of Eventually Consistent / CRDT

- With A under P because of very weak C
- In addition, with **S** because of very weak C
 - The coordination used for upkeeping consistency (e.g., Paxos, Raft) has been the bottleneck of throughput and scalability
 - E.g., needs to 'wait-for' (e.g., 3 out of 5)
 - 90% of time is spent on **waiting for coordination**
- EC Systems are *coordination-free*!
 - No coordination != No communication
 - CRDT still needs communication to exchange and merge
 - No-coordination = wait-free